

Waygr v1 — Build brief

Build Waygr in phases. Phase 1 is an installable PWA on Vercel (Next.js, Supabase) that runs the full one-on-one loop: create a challenge, accept from a link with no install, watch it live, settle from the score feed, pay the forfeit, rematch. Nothing in Phase 1 gets thrown away later. This doc is self-contained; PRODUCT-BRIEF.md, GROWTH-LOOP.md and BRAND-BRIEF.md explain the why.

Phases

Phase	Goal	Adds	Done when
1. Beta (Sep 28 – Nov 13, 2026)	The 1v1 loop works for 50 friends	Auth, games list, create flow, challenge landing with link preview, accept, live page, trash talk, settlement, forfeits, result cards, rematch, rivalry record, web push, analytics, installable PWA	Acceptance criteria below pass; dogfood on NFL Sundays Nov 15 and 22; beta opens Nov 26
2. Groups and rooms (Dec 2026 – Jan 2027)	One link pulls in a whole chat or room	Group challenges ("who's in?"), counter-offers on the line, rooms with a QR code, room and group leaderboards	A 10-person room runs a full game without help
3. Native (Feb – Mar 2027)	Live on the lock screen for March Madness	Capacitor shell around the same app, iMessage extension, Live Activities, widgets, haptics, App Store listing	Challenge created and accepted inside iMessage

Locked decisions

Decision	Choice	Why

Platform	PWA first: Next.js 15 App Router, TypeScript, Tailwind, Serwist, on Vercel	A link opens it instantly; native comes in Phase 3
Backend	Supabase: Postgres, Auth, Realtime, Storage, Edge Functions, Cron. Drizzle for schema and migrations	One service; same setup as Standfast
Stakes	Forfeits only: concession card, jersey swap, custom. No money, drinks, prizes or payment links anywhere	See Guardrails in the Product brief
Who takes the other side	The first signed-in person, other than the creator, to tap "I'm in" on the link	Works in 1:1 texts and group chats with no contact picker
Lines	The creator sets spread and total lines in 0.5 steps; the stepper defaults to a half point	No sportsbook odds; half points avoid most pushes
Lock	Every challenge locks at kickoff; unaccepted ones expire	One simple rule
Leagues in Phase 1	NFL, college football, NBA. College basketball added before March	Covers Thanksgiving, bowls and the playoffs
Referee	The score feed only. No self-reporting; an admin can void	No disputes
Score feed	Behind a <code>ScoreProvider</code> interface. Start with BALLDONTLIE; SportsDataIO is the fallback	Swap providers without touching settlement
Polling	Supabase Cron calls the <code>poll-scores</code> Edge Function every 30 s	Vercel Hobby cron only runs once a day ; Supabase Cron runs down to every second
Cards and previews	<code>next/og</code> image routes, rendered on request and cached	Every preview shows the real matchup
Analytics	PostHog, free tier	Funnels for accept rate and k-factor out of the box
Naming	App name in one constant (<code>APP_NAME</code>);	Renames and copy edits touch

and copy	every UI string in <code>lib/copy.ts</code>	one file
Theme	Tokens from the Brand brief; dark by default, follows the system setting	Every pair already checked for contrast

Definitions and rules

- **Challenge:** one call on one game between the creator and one opponent, with one forfeit.
- **Call:** a market, the creator's pick and, for spread and total, a line. The opponent always holds the opposite pick.
- **Forfeit:** what the loser owes. It is created at settlement and is either owed or paid.
- **Rivalry:** the running record between two people across all their settled challenges.

Market	Creator picks	Settles when	Creator wins if	Push if	Leagues
Game winner	Home or away	Final	Their team wins	The game ends tied	All
Spread	A team and a line, such as KC -3.5	Final	Team score + line > other score	Team score + line = other score	All
Total	Over or under a line	Final	Combined score is on their side of the line	Combined score = line	All
Halftime leader	Home or away	End of the first half	Their team leads at the half	Tied at the half	All
Quarter winner	Home or away, plus quarter 1-4	End of that quarter	Their team outscored the other in that quarter	Quarter scoring is equal	NFL, NCAAF, NBA (not men's college basketball, which plays halves)

Overtime counts toward game winner, spread and total. A game that is postponed, suspended or canceled voids every challenge on it. A game with no final from the feed 12 hours after its scheduled start gets flagged to the admin, who can void it.

Settlement is one pure function, `settle(challenge, game)`, returning `creator`, `opponent`, `push` or `null` (not decidable yet). It never writes anything. The settle job calls it and writes the result in one transaction, and running it twice on the same game changes nothing.

```
stateDiagram-v2
    [*] --> open: created
    open --> accepted: someone taps I'm in
    open --> canceled: creator cancels
    open --> expired: kickoff, not accepted
    accepted --> live: game starts
    accepted --> void: game postponed or canceled
    live --> settled: settle() returns a result
    live --> void: game suspended or canceled
    settled --> [*]
    void --> [*]
    expired --> [*]
    canceled --> [*]
```

Accepting runs as one conditional update (where `state = 'open'` and `kickoff > now()` and `creator_id <> :user`). If two people tap at once, exactly one wins; the other sees "Too slow. Jordan took it." and a button to make their own call on the same game.

Data model

Postgres on Supabase, defined in Drizzle. Times are `timestampz` in UTC and shown in the viewer's local time.

```
create type league as enum ('nfl','ncaaf','nba','ncaab');
create type game_status as enum ('scheduled','live','final','postponed','suspende
create type market as enum ('winner','spread','total','half_leader','quarter_winn
create type pick as enum ('home','away','over','under');
create type challenge_state as enum ('open','accepted','live','settled','void','e
create type outcome as enum ('creator','opponent','push');
create type forfeit_kind as enum ('concession','jersey_swap','custom');
create type forfeit_status as enum ('owed','proof_submitted','paid');

create table profiles (
  id uuid primary key default gen_random_uuid(),
  auth_user_id uuid unique references auth.users on delete set null, -- null once
  handle text unique not null, -- from the sign-in name, editable
```

```

display_name text not null,          -- becomes 'Deleted user' on account
avatar_url text,
adult_confirmed_at timestamptz,      -- set by the 18+ checkbox; required
referred_by uuid references profiles(id), -- creator of the first challenge thi
jersey_team text,                    -- team code while a jersey-swap forf
jersey_until timestamptz,
created_at timestamptz not null default now(),
deleted_at timestamptz
);

create table teams (
  code text primary key,              -- e.g. 'nfl:KC'
  league league not null,
  abbr text not null, name text not null,
  primary_color text not null, secondary_color text not null
);

create table games (
  id uuid primary key default gen_random_uuid(),
  provider text not null, provider_game_id text not null,
  league league not null,
  home_team text not null references teams(code),
  away_team text not null references teams(code),
  starts_at timestamptz not null,
  status game_status not null default 'scheduled',
  period int, clock text,
  home_score int not null default 0, away_score int not null default 0,
  period_scores jsonb not null default '[]', -- [{"period":1,"home":7,"away":3},
  updated_at timestamptz not null default now(),
  unique (provider, provider_game_id)
);

create table challenges (
  id uuid primary key default gen_random_uuid(),
  slug text unique not null,          -- 10-char nanoid, the public link
  creator_id uuid not null references profiles(id),
  opponent_id uuid references profiles(id), -- null until accepted
  game_id uuid not null references games(id),
  market market not null,
  creator_pick pick not null,
  line numeric(5,1),                  -- spread (from the creator's team) o
  quarter int check (quarter between 1 and 4),
  forfeit_kind forfeit_kind not null,
  forfeit_text text,                  -- custom text, screened on write
  state challenge_state not null default 'open',
  outcome outcome,
  rematch_of uuid references challenges(id),

```

```

    created_at timestamptz not null default now(),
    accepted_at timestamptz, settled_at timestamptz,
    check (opponent_id is null or opponent_id <> creator_id)
);
create index on challenges (game_id, state);

create table forfeits (
    id uuid primary key default gen_random_uuid(),
    challenge_id uuid unique not null references challenges(id),
    owed_by uuid not null references profiles(id),
    owed_to uuid not null references profiles(id),
    kind forfeit_kind not null,
    status forfeit_status not null default 'owed',
    proof_path text, -- Supabase Storage path for custom p
    due_at timestamptz not null, -- settled_at + 7 days
    paid_at timestamptz
);

create table messages ( -- trash talk, one thread per challen
    id bigint generated always as identity primary key,
    challenge_id uuid not null references challenges(id) on delete cascade,
    author_id uuid not null references profiles(id) on delete cascade,
    body text not null check (char_length(body) <= 280),
    created_at timestamptz not null default now()
);

create table push_subscriptions (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references profiles(id) on delete cascade,
    endpoint text unique not null, p256dh text not null, auth text not null,
    created_at timestamptz not null default now()
);

create table notifications_sent ( -- enforces caps and makes sends idem
    user_id uuid not null references profiles(id) on delete cascade,
    kind text not null, ref_id uuid not null,
    sent_at timestamptz not null default now(),
    primary key (user_id, kind, ref_id)
);

create table blocks (
    blocker_id uuid references profiles(id) on delete cascade,
    blocked_id uuid references profiles(id) on delete cascade,
    primary key (blocker_id, blocked_id)
);

create table reports (

```

```

    id bigint generated always as identity primary key,
    reporter_id uuid not null references profiles(id),
    challenge_id uuid references challenges(id),
    message_id bigint references messages(id) on delete set null,
    reason text not null,
    created_at timestamptz not null default now()
);

-- Head-to-head record between any two people, from settled challenges
create view rivalries as
select least(creator_id, opponent_id) as user_a,
       greatest(creator_id, opponent_id) as user_b,
       count(*) filter (where (outcome='creator' and creator_id < opponent_id)
                        or (outcome='opponent' and opponent_id < creator_id)) as
count(*) filter (where (outcome='creator' and creator_id > opponent_id)
                or (outcome='opponent' and opponent_id > creator_id)) as
count(*) filter (where outcome='push') as pushes,
       max(settled_at) as last_settled_at
from challenges where state='settled'
group by 1, 2;

```

This schema loads cleanly on Postgres 16, and the rivalry view was checked against sample challenges. Row-level security keys off `profiles.auth_user_id = auth.uid()` : people read and write only their own profile, subscriptions and blocks. A challenge is readable by its creator and opponent; the public landing page reads it server-side with the service role and exposes only display fields. Messages are readable by the two people in the challenge and writable by them until 24 hours after settlement. Only Edge Functions (service role) write `games` , challenge state after acceptance, and forfeits.

Deleting an account removes the auth user, messages, push subscriptions and proof files, and renames the profile to "Deleted user." Settled challenges stay, so the other person's record doesn't change.

Storage: bucket `proof` , private, max 50 MB per file, images and videos up to 30 seconds. Signed URLs expire after 1 hour; files are deleted 90 days after the forfeit is paid.

Routes and jobs

Route or job	What it does
GET /c/[slug]	The challenge page, server-rendered for everyone. Its state (open, taken, live, settled, void) decides the view. No sign-in

	needed to see it
GET /c/[slug]/opengraph-image	The 1200 × 630 link preview: matchup, call, forfeit, both avatars. Cached until the challenge changes state
GET /api/cards/[slug]? type=called concession proof&format=post story	Result cards as PNG, 1080 × 1350 or 1080 × 1920
POST /api/challenges	Create: game, market, pick, line, quarter, forfeit. Returns the slug. Rejects games that start in under 2 minutes
POST /api/challenges/[id]/accept	The conditional update from Definitions. First accept by a new user sets <code>referred_by</code>
POST /api/challenges/[id]/cancel	Creator only, while open
POST /api/challenges/[id]/rematch	Clones opponent, market and forfeit onto the next game of either team; prefilled, still editable
POST /api/forfeits/[id]/paid	Concession: called after the share sheet resolves
POST /api/forfeits/[id]/proof	Custom: upload to Storage, status becomes <code>proof_submitted</code>
POST /api/forfeits/[id]/confirm and /reject	Winner confirms or rejects proof. Auto-confirms after 72 hours
GET /api/games?league=&from=&to=	Games for the create flow, next 7 days
POST /api/push/subscribe and DELETE	Web push subscription
POST /api/account/delete	As described under the data model
Edge Function <code>poll-scores</code>	Supabase Cron, every 30 s. Updates games that are live or start within 15 minutes and have an accepted challenge. Every 10

	minutes, refreshes the next 7 days of schedules
Edge Function <code>settle</code>	Runs after each poll for games that changed. Moves accepted challenges to live when their game starts. Calls <code>settle()</code> on every accepted or live challenge, writes outcomes and forfeits in one transaction, queues notifications
Edge Function <code>sweep</code>	Every minute: expires open challenges past kickoff, voids challenges on postponed or canceled games, auto-confirms proof after 72 hours, ends jersey frames

Trash talk reads and writes `messages` straight through Supabase with row-level security, and the live page subscribes to `games` and `messages` changes through Supabase Realtime.

Score feed

Every provider sits behind one interface, so switching providers never touches settlement:

```
interface ScoreProvider {
  listGames(league: League, from: Date, to: Date): Promise<GameUpsert[]>;
  getLive(providerGameIds: string[]): Promise<GameUpdate[]>;
}
// GameUpdate: status, period, clock, homeScore, awayScore,
// periodScores [{ period, home, away }] with overtime as period 5+
```

Start with [BALLDONTLIE](#), which covers NFL, NBA and both college sports and prices per sport (\$0 to \$39.99 a month). Before paying, confirm the chosen tier returns live scores and period-by-period scores for NFL and college football. If it doesn't, use [SportsDataIO](#) instead.

Adapter rules: map provider statuses onto `game_status`. During halftime and between quarters, report `period` as the next period, so finished periods settle at the break. Poll only games people have challenges on.

Settlement

This reference implementation passes 24 test cases, attached as `settle.test.ts` (Vitest; both files type-check under strict TypeScript): every market, pushes, overtime, live-period settling and the college-basketball exceptions. Ship both files as-is and keep the tests passing.

```
export type Market = 'winner' | 'spread' | 'total' | 'half_leader' | 'quarter_win
export type Pick = 'home' | 'away' | 'over' | 'under';
export type Outcome = 'creator' | 'opponent' | 'push';
export type GameStatus = 'scheduled' | 'live' | 'final' | 'postponed' | 'suspende

export interface Game {
  league: 'nfl' | 'ncaaf' | 'nba' | 'ncaab';
  status: GameStatus;
  period: number | null; // current period; for a final game, the last period pla
  homeScore: number;
  awayScore: number;
  periodScores: { period: number; home: number; away: number }[]; // 5+ = overtim
}

export interface Challenge {
  market: Market;
  creatorPick: Pick;
  line: number | null; // spread from the creator's team, or the total
  quarter: number | null; // 1-4 for quarter_winner
}

/** Pure. Returns the result, or null while it can't be decided yet. Void is hand
export function settle(c: Challenge, g: Game): Outcome | null {
  const final = g.status === 'final';
  const side = (home: number, away: number): Outcome => {
    if (home === away) return 'push';
    const homeAhead = home > away;
    return (c.creatorPick === 'home') === homeAhead ? 'creator' : 'opponent';
  };
  // Is period p complete? True once the game is final or play has moved past it.
  const periodDone = (p: number) => final || (g.status === 'live' && (g.period ??
  const scoreThrough = (last: number) =>
    g.periodScores.filter(s => s.period <= last)
      .reduce((a, s) => ({ home: a.home + s.home, away: a.away + s.away })), { hom
  const halfEnds = g.league === 'ncaab' ? 1 : 2; // men's college hoops plays two

  switch (c.market) {
    case 'winner':
      return final ? side(g.homeScore, g.awayScore) : null;
    case 'spread': {
      if (!final || c.line === null) return null;
```

```

    const mine = c.creatorPick === 'home' ? g.homeScore : g.awayScore;
    const theirs = c.creatorPick === 'home' ? g.awayScore : g.homeScore;
    const margin = mine + c.line - theirs;
    return margin === 0 ? 'push' : margin > 0 ? 'creator' : 'opponent';
  }
  case 'total': {
    if (!final || c.line === null) return null;
    const total = g.homeScore + g.awayScore;
    if (total === c.line) return 'push';
    return (c.creatorPick === 'over') === (total > c.line) ? 'creator' : 'oppon
  }
  case 'half_leader': {
    if (!periodDone(halfEnds)) return null;
    const h = scoreThrough(halfEnds);
    return side(h.home, h.away);
  }
  case 'quarter_winner': {
    if (c.quarter === null || g.league === 'ncaab' || !periodDone(c.quarter)) r
    const q = g.periodScores.find(s => s.period === c.quarter);
    return q ? side(q.home, q.away) : null;
  }
}
}
}

```

On a result: set `outcome`, `state = 'settled'` and `settled_at`. If someone lost, create the forfeit (`due_at = settled + 7 days`). A jersey swap is marked paid immediately and sets the loser's `jersey_team` and `jersey_until` (7 days). A push creates no forfeit. Everything happens in one transaction guarded by `where state in ('accepted', 'live')`, so a second run is a no-op.

Screens and flows

The two flows that matter most, with hard limits:

Accept (new user, from a text). Open link → see the challenge → tap "I'm in" → sign in with Apple, Google or phone code (one sheet, plus the 18+ checkbox the first time) → "You're on." At most 3 taps plus the sign-in provider's own screen. The page is readable before any sign-in. Right after accepting, show "Your turn" with three one-tap calls on tonight's games.

Create (returning user). "Make a call" → pick a game → pick market, side and line → pick a forfeit → preview → native share sheet. Under 20 seconds. Defaults are preselected at every step (spread, the favorite, the last forfeit used), so a fast user just taps Next.

--	--	--

Screen	Route	Contents
Challenge	/c/[slug]	One route, five views. Open: the call, both sides, forfeit, "I'm in" / "Not this one." Taken: "Too slow. Jordan took it." plus "Make your own call." Live: score strip, challenge meter, status line, trash talk, rivalry line. Settled: "Called it." or "Not your night," forfeit action, Rematch, Share. Void: reason and "Make a new call"
Sign-in sheet	Overlay	Apple, Google, phone code; 18+ checkbox; links to terms and privacy
Home	/	Live challenges first, then owed forfeits, then open challenges waiting on someone, then tonight's games with a one-tap call on each. Orange "Make a call" pinned at the bottom
Create	Bottom sheet	Game list grouped by day and league with team colors, then market picker, side, line stepper (0.5 steps), then forfeit: concession, jersey swap or custom (80 characters max, screened)
Forfeit	/f/[id]	Loser: share the concession card, or record and upload proof. Winner: confirm or reject proof
Profile	/u/[handle]	Avatar with jersey frame if active, record (W-L-P), forfeit paid rate, "Owes N," top rivalries
Rivalry	/r/[handle]	Head-to-head record and every challenge between the two of you, newest first
Settings	/settings	Notifications, add to home screen instructions, blocked people, sign out, delete account

Screen rules:

- **Install prompt:** never before the first accept. After the first settled challenge, show one card: "Get pinged when you win. Add Waygr to your home screen." On iPhone, show the three steps with the Share icon. iOS web push only works once the app is on the home screen.
- **Custom forfeit screening:** reject text that mentions alcohol, drinking, money, dollar amounts, payment apps or anything dangerous, with the message "Keep it legal and sober. Try a jersey or a public apology." Keep the term list in `lib/forfeit-screen.ts`.
- **Blocking:** blocked people can't take your open challenges; their messages are hidden

from you.

- **Report:** a report option on every message and custom forfeit, logged to `reports`.
- **Loading and failure states:** skeletons, never spinners, for the score strip. If the feed is more than 2 minutes stale on a live game, show "Score's lagging. Retrying."

Notifications

Web push only, sent from Edge Functions. Every send is logged in `notifications_sent` first, so none goes out twice. Waygr never texts or emails anyone on its own.

Trigger	To	Copy	Limit
Challenge accepted	Creator	Jordan's in. Chiefs -3.5, Sunday 3:25.	Once
15 minutes to kickoff	Both	Chiefs-Bills in 15. You've got Chiefs -3.5.	Once per challenge
Lead change on the call	Both	You're up. KC covering by 3.5.	3 per challenge
Settled	Both	Called it. / Not your night. Rematch? / Push. Nobody owes.	Once
Proof submitted	Winner	Jordan paid up. Check the proof.	Once
Forfeit reminder	Loser	You owe Sam one.	At 24 hours and on day 6
Rivalry nudge	Both	Sam leads you 4-3. Chiefs play Sunday.	Once a week per person

Hard caps: 6 pushes per person per day. Quiet hours run 11 pm to 9 am local, except for games the person has a live challenge on.

Analytics events

PostHog, fired from the server where possible so ad blockers don't break the funnel. The Growth loop metrics are built from these, so names and properties are fixed.

Event	Properties	Feeds

link_opened	challenge_id, is_signed_in, challenge_state	Accept rate
signin_completed	method, is_new_user, from_challenge_id	Time to "You're on"
challenge_accepted	challenge_id, is_new_user, seconds_since_open, creator_id	Accept rate, time to accept, k-factor
challenge_created	challenge_id, league, market, forfeit_kind, is_rematch, seconds_in_flow	Activation
challenge_shared	challenge_id, method (share_sheet, copy_link)	Invite volume
challenge_settled	challenge_id, market, outcome	Rematch rate, share rate
card_shared	challenge_id, card_type, format	Share rate
forfeit_paid	forfeit_id, kind, hours_after_settle	Forfeit paid rate
rematch_created	challenge_id, rematch_of	Rematch rate
pwa_installed	platform	Push reach
push_enabled	platform	Push reach

k-factor for a week = new users that week with a `referred_by` ÷ people who created at least one challenge that week.

Non-goals for Phase 1

- Money in any form: payments, payment links, prizes, sportsbook odds.
- Matching strangers or a public feed of challenges.
- Group challenges, rooms, leaderboards and counter-offers (Phase 2).
- Native app, iMessage extension, Live Activities, widgets and haptics (Phase 3).
- Importing contacts, or Waygr sending texts or emails to anyone.
- Chat outside a challenge.
- Leagues beyond NFL, college football and NBA (college basketball comes before March).

Acceptance criteria

- On a real iPhone (Safari) and Android phone (Chrome), a signed-out person opens a challenge link from iMessage or WhatsApp and reaches "You're on." in 3 taps plus the sign-in provider's screen.
- iMessage and WhatsApp link previews show the challenge image, not a generic card.
- The challenge page loads its main content in under 2.0 s on a mid-range phone over 4G (Lighthouse mobile LCP).
- Nobody can accept their own challenge, accept after kickoff, or accept one already taken. Two simultaneous accepts produce exactly one opponent (tested).
- `settle.test.ts` passes all 24 cases. A replay test runs a recorded full NFL game through `poll-scores` and `settle` and gets every market right.
- Challenges settle within 2 minutes of the feed reporting a final or a finished period. Running settle twice creates no duplicate forfeits or notifications.
- A postponed or canceled game voids its challenges; unaccepted challenges expire at kickoff.
- The live page updates score and meter within 60 s without a refresh; a trash-talk message shows for the other person within 2 s.
- The concession card opens the native share sheet with the image on iOS and Android and marks the forfeit paid when sharing completes; desktop falls back to a download.
- The jersey frame shows on the loser's avatar for 7 days, then disappears.
- Custom proof uploads, the winner can confirm or reject, and it auto-confirms after 72 hours.
- Custom forfeit screening blocks alcohol, money and payment-app terms.
- A test fails the build if any string in `lib/copy.ts` contains the whole words "bet," "wager," "odds" or "payout."
- The 18+ checkbox is required before a person's first create or accept.
- Colors come only from the Brand brief tokens. axe reports no serious or critical issues on Challenge, Home and Create in both themes, and layouts hold at 200% text size.
- Lighthouse installability passes, and web push arrives on an iPhone after adding Waygr to the home screen.
- Push caps and quiet hours are enforced (unit tested).

- Every analytics event fires once, with its listed properties, in PostHog's live events view.
- Deleting an account works as specified and leaves the other person's record unchanged.
- Through the Supabase client, a signed-in person can't read challenges or messages they're not part of (tested).

Delivery

- Repo: Next.js app, drizzle/ migrations, supabase/functions/ (poll-scores, settle, sweep), lib/settle.ts and lib/settle.test.ts (attached), lib/copy.ts , lib/forfeit-screen.ts , styles/tokens.css (from the Brand brief), fixtures/nfl-full-game.json for the replay test, docs/ with these four docs and waygr-brand-preview.png , .cursor/rules/waygr.mdc summarizing Locked decisions, Non-goals and the color rules. README covers Supabase setup, auth providers, VAPID key generation, Supabase Cron entries and the replay script.
- Env vars: NEXT_PUBLIC_APP_URL , NEXT_PUBLIC_SUPABASE_URL , NEXT_PUBLIC_SUPABASE_ANON_KEY , SUPABASE_SERVICE_ROLE_KEY , DATABASE_URL , NEXT_PUBLIC_VAPID_PUBLIC_KEY , VAPID_PRIVATE_KEY , SCORE_PROVIDER , BALLDONTLIE_API_KEY , NEXT_PUBLIC_POSTHOG_KEY , NEXT_PUBLIC_POSTHOG_HOST .
- Vercel: production project plus a preview deploy per PR. Supabase: separate dev and prod projects.

Questions for Chris

Ask only these; for anything else not covered here, pick the simplest option that meets the acceptance criteria and note it in the README.

1. Which domain do challenge links use?
2. Is there an Apple Developer account? Sign in with Apple on the web needs one (\$99 a year), and Phase 3 needs it anyway.
3. After confirming what the BALLDONTLIE tiers return, which tier should we pay for?

Prompt to paste

You are building Waygr v1, an installable PWA (Next.js 15 App Router, TypeScript,

Work in this order, one PR each, and run the tests before telling me a step is do

1. Scaffold, tokens.css, copy.ts with the banned-words test, schema, migrations,
2. ScoreProvider adapter for BALLDONTLIE, the poll-scores, settle and sweep Edge
3. Auth (Apple, Google, phone), the 18+ gate, the challenge page with all five vi
4. Games API, the create sheet, Home, cancel and rematch.
5. Live view with Realtime scores, the challenge meter and trash talk.
6. Forfeits (concession share, jersey frame, custom proof), result cards, Profile
7. PWA install card, service worker, web push with caps and quiet hours, and ever
8. Settings, block and report, delete account, README.

After each step, list which acceptance criteria now pass. Do not add features tha